

AUV ZIT6 固件系统

详细设计与实现文档

ZIT6 开发团队

2026 年 7 月 2 日

目录

1 项目概述	3
1.1 项目简介	3
1.2 硬件平台	3
1.3 软件栈	3
1.4 目录结构	4
2 Safety —安全监控	4
2.1 SafetyMonitor	4
2.1.1 ARM / DISARM 逻辑	4
2.1.2 解锁条件	5
2.1.3 上锁条件	5
2.2 SoftWatchdog	5
3 Peripherals —外设驱动	5
3.1 INS_Driver (NAV-300 惯导)	5
3.2 MotionController_Driver (动力控制板)	6
3.3 USBL_Driver (超短基线定位)	6
3.4 MS5837_Driver (深度传感器)	6
3.5 External Drivers	6
4 Porting —硬件适配层	6
4.1 函数指针解耦模式	7
4.2 Porting 实现	7
5 Common —全局上下文	7
5.1 AppContext —依赖注入容器	7
5.2 MotionContext —运动状态共享	8
5.3 SystemContext —系统状态	8
5.4 LockedField —线程安全字段	8
6 Application —任务层	8
6.1 ControlTask (100Hz 控制主循环)	8
6.2 MicroRosTask (通信任务)	9
6.3 MonitorTask (10Hz 安全监控)	9

7 通信协议	10
7.1 控制指令—ZitSetpoint	10
7.1.1 消息结构	10
7.1.2 control_key	10
7.2 状态发布	11
7.3 订阅指令	11
7.4 参数服务	11
8 配置系统	11
8.1 config.json → C++ 结构体	11
8.2 全局配置实例	12
9 坐标系定义	12
10 仿真支持	13
10.1 SITL (Software-in-the-Loop)	13
10.2 HITL (Hardware-in-the-Loop)	13

1 项目概述

1.1 项目简介

AUV ZIT6 是一款基于 STM32H7 系列微控制器的自主水下航行器 (Autonomous Underwater Vehicle, AUV) 固件系统。项目采用现代 C++17 模块化架构设计, 实现了 6-DOF (六自由度) 级联控制, 支持位置环、速度环和直接推力控制三种工作模式。

1.2 硬件平台

- **微控制器:** STM32H743ZI (ARM Cortex-M7, 双精度 FPU)
- **通信外设:**
 - USART2 —micro-ROS (DDS 协议, 与上位机通信)
 - UART4 —VIT6 下位机 (推进器控制板)
 - UART7 —NAV-300 惯导
 - USART1 —USBL 超短基线定位系统
 - I2C1/2 —MS5837 深度传感器
- **传感器:** NAV-300 惯导 (INS)、MS5837 压力传感器、DVL 多普勒测速仪
- **操作系统:** FreeRTOS (CMSIS-RTOS V2 封装)

1.3 软件栈

组件	技术选型
微控制器	STM32H743ZI (ARM Cortex-M7)
RTOS	FreeRTOS + CMSIS-RTOS V2
通信	micro-ROS (基于 DDS/UXR-CE)
编程语言	C++17, 部分 C
数值计算	Eigen 3.4+ (仅 Core, 零动态内存)
构建系统	CMake 3.22+ (STM32CubeMX 集成)
调试	SEGGER RTT / J-Link / UART
推进器协议	定长帧 FA AF [ID] [Payload] FB BF

表 1: 软件栈概览

1.4 目录结构

Listing 1: 项目顶层目录结构

```
1 UserApp/
2   Algorithm/           # 纯数值算法（无 RTOS 依赖）
3   Application/         # FreeRTOS 任务层
4   Common/              # 全局上下文与工具类
5   Component/           # 高内聚功能组件
6     Chassis/           # 底盘控制
7     ConfigService/     # 参数配置服务
8     HitlSimulator/     # HIL 仿真器
9     MicroRos/          # micro-ROS 封装
10    RosLogger/          # 日志队列
11    Safety/             # 安全监控
12  Peripherals/         # 外设驱动与 HAL
13  Porting/              # 硬件适配层
14  Config/               # 配置管理
```

2 Safety —安全监控

2.1 SafetyMonitor

独立安全监控模块，从 ControlTask 中抽离，在 MonitorTask 中以 10Hz 运行。

2.1.1 ARM / DISARM 逻辑

参数	值
最小解锁心跳数	10 次
最小解锁持续时间	1000 ms
已解锁心跳超时	500 ms
未解锁心跳超时清零	1000 ms
PWM 输出使能	推力 [-1.0, 1.0] 映射为 [-1000, 1000]

表 2: 安全监控参数

2.1.2 解锁条件

1. 心跳计数 ≥ 10 且持续 ≥ 1000 ms
2. `last_heartbeat_data == 1` + 导航有效, 或 `data == 3` 跳过导航检查
3. 解锁时自动记录 Home Offset (当前位置为原点)

2.1.3 上锁条件

1. 已解锁状态下 500 ms 内未收到心跳 $\rightarrow$ 强制上锁
2. 上锁时清空 Home Offset, 控制层级切回 NONE

2.2 SoftWatchdog

软件看门狗, 监控三个组件的心跳状态:

- MICROROS —micro-ROS 通信状态
- INS —惯导数据接收状态
- DEPTH —深度传感器数据状态

所有组件正常时, MonitorTask 喂硬件看门狗 (IWDG)。

3 Peripherals —外设驱动

3.1 INS_Driver (NAV-300 惯导)

采用函数指针接口解耦硬件操作:

- InsPortOps 结构体封装 `init/read/transmit` 三个函数指针
- 字节流解析, 帧长 133-256 字节
- 输出 `MotionContext::NavState` (6-DOF 位姿 + 6-DOF 速度)
- 支持命令: DVL 开关、重启、位置重置、经纬度装订

3.2 MotionController_Driver (动力控制板)

通过 VIT6 下位机通信协议控制执行器：

Listing 2: 协议数据包格式

```
1 struct ThrustPacket {  
2     uint8_t head[2];    // FA AF  
3     uint8_t id;         // 0x01(推力)  
4     float Fx, Fy, Fz;   // 三轴力 (N)  
5     float Fyaw, Fpitch, Froll; // 三轴力矩  
6     uint8_t tail[2];    // FB BF  
7 };
```

支持命令 ID: 0x01 (推力)、0x02 (舵机)、0x03 (灯光)、0x07 (推力曲线)

3.3 USBL_Driver (超短基线定位)

通过 USART1 + DMA 接收 USBL 数据：

- 帧头 EB BE，帧尾 ED DE
- 133 字节定长帧，异或校验
- 输出：信标北向/东向/深度、基阵姿态、信号能量

3.4 MS5837_Driver (深度传感器)

支持 I2C 和 UART 两种通信后端（通过 DepthCalcBoard 解算板），提供深度和温度测量。

3.5 External Drivers

- **HAL**: STM32CubeMX 生成的 HAL 驱动 (I2C、UART、DMA、IWDG、TIM)
- **FreeRTOS**: CMSIS-RTOS V2 封装，heap_4 内存管理

4 Porting — 硬件适配层

Porting 层封装具体硬件引脚和中断服务，提供 **函数指针接口表** 供上层驱动使用。

4.1 函数指针解耦模式

驱动类（如 INS_Driver）的构造函数接收 PortOps 结构体，运行时绑定具体硬件实现：

Listing 3: PortOps 接口模式示例

```
1 struct InsPortOps {
2     void *ctx;
3     bool (*init)(void *ctx);
4     uint16_t (*read)(void *ctx, uint8_t *buf, uint16_t max_len);
5     bool (*transmit)(void *ctx, const uint8_t *data, uint16_t len);
6 };
7
8 struct MotorPortOps {
9     void *ctx;
10    bool (*transmitDMA)(void *ctx, const uint8_t *data, uint16_t size
11        );
12    void *(*getTxPacket)(void *ctx);
13 };
```

4.2 Porting 实现

- **INS_Porting**: UART7 DMA 实现 INS_Driver 的 InsPortOps
- **MotionController_Porting**: UART4 DMA 实现 MotionController_Driver 的 MotorPortOps
- **MS5837_Porting**: I2C/UART 实现深度传感器后端
- **DepthCalcBoard_Porting**: 深度解算板通信适配

5 Common —全局上下文

5.1 AppContext —依赖注入容器

使用结构体聚合所有驱动和组件指针，通过构造函数注入任务：

Listing 4: AppContext 定义

```
1 struct AppContext {
```



```
2     INS_Driver *ins_driver;  
3     MotionController_Driver *motor_driver;  
4     MS5837_Driver *depth_sensor;  
5     RosLogger *logger;  
6     SoftWatchdog *watchdog;  
7     ChassisManager *chassis;  
8 };  
9 extern AppContext g_app_ctx;
```

5.2 MotionContext —运动状态共享

跨任务共享的运动状态，所有字段使用 LockedField<T> 临界区保护：

- nav_state_ —导航状态（6-DOF 位姿 + 6-DOF 速度）
- current_setpoint_ —当前目标设定值
- last_output_forces_ —6-DOF 输出力快照
- home_offset_ —解锁原点偏移

5.3 SystemContext —系统状态

- arm_state_ —解锁/上锁状态 + 心跳计数
- nav_status_ —惯导/DVL 状态标志
- is_planner_active —规划器启用标志

5.4 LockedField —线程安全字段

模板类,对任意可拷贝类型 T 自动加 taskENTER_CRITICAL() / taskEXIT_CRITICAL():

- get() —临界区读取
- set(v) —临界区写入

6 Application —任务层

6.1 ControlTask (100Hz 控制主循环)

Listing 5: ControlTask 主循环

```

1 void ControlTask::run() {
2     init();
3     for (;;) {
4         updateNavigation();    // 1. 获取 INS / SITL / HIL 导航数据
5         computeAndPublish();   // 2. 执行底盘控制 + 推力输出
6         vTaskDelayUntil(&last_wake_time_, pdMS_TO_TICKS(10));
7     }
8 }

```

updateNavigation() 数据源选择:

- HITL 模式: 由 HitlSimulator 提供
- SITL 模式: 由 /zit6/sim/nav 话题队列提供
- 实物模式: INS_Driver::getNavState(), 并融合 MS5837 深度

6.2 MicroRosTask (通信任务)

封装 micro-ROS 初始化、发布、订阅和服务:

Listing 6: MicroRosTask 结构

```

1 class MicroRosTask {
2     MicroRosTransport transport_;
3     MicroRosPublisher publisher_;
4     MicroRosSubscriber subscriber_;
5     MicroRosService service_;
6 };

```

6.3 MonitorTask (10Hz 安全监控)

Listing 7: MonitorTask 主循环

```

1 void MonitorTask::run() {
2     init();
3     for (;;) {
4         refreshHardwareWatchdogIfNeeded(); // 软狗检查 $\rightarrow$
           喂硬狗
5         safety_monitor_.check(HAL_GetTick()); // ARM/DISARM 状态机

```

```
6         vTaskDelayUntil(&last_wake_time_, pdMS_TO_TICKS(100));
7     }
8 }
```

7 通信协议

7.1 控制指令—ZitSetpoint

7.1.1 消息结构

Listing 8: ZitSetpoint 消息定义

```
1 # zit6_interfaces/msg/ZitSetpoint.msg
2 uint8 control_key    # 控制模式 + 标志位
3 uint8 type_mask      # 轴掩码
4 float32 x y z        # 目标值 (m/m/s/N)
5 float32 yaw          # 偏航 (rad/rad/s)
6 uint32 seq           # 序列号 (可选)
```

7.1.2 control_key

位域	含义
Bit 0-1	模式: 0=POSITION, 1=VELOCITY, 2=FORCE
Bit 4 (0x10)	机体系坐标 (默认世界系 NED)
Bit 5 (0x20)	增量模式

表 3: control_key 定义

7.2 状态发布

话题	内容	频率
/zit6/state/pos	NED 位置 $[x, y, z, yaw]$	30Hz
/zit6/state/vel	FRD 速度 $[vx, vy, vz, vyaw]$	50Hz
/zit6/state/thr	归一化推力 $[Fx, Fy, Fz, Myaw]$	30Hz
/zit6/state/status	系统状态 (ARM、层级、INS)	10Hz
/zit6/state/zithbt	节点心跳	1Hz
/zit6/log	ROS2 日志	事件驱动

表 4: 状态发布话题

7.3 订阅指令

- /zit6/cmd/setpoint — 控制设定点 (ZitSetpoint)
- /zit6/cmd/agxhbt — 上位机心跳 (UInt32, 用于 ARM)
- /zit6/cmd/ins — 惯导指令 (UInt8)
- /zit6/cmd/servo — 舵机角度 (Float32)
- /zit6/cmd/light — LED 灯控 (UInt8)
- /zit6/sim/nav — SITL 仿真导航 (Float32MultiArray[12])

7.4 参数服务

- /zit6/update_params — JSON 格式实时更新参数
- /zit6/get_params — 按路径查询参数

8 配置系统

8.1 config.json → C++ 结构体

配置通过 gen_config.py 脚本生成 C++ 头文件:

Listing 9: 配置结构体示例

```

1 struct AxisConfig {
2     float pos_kp, pos_ki, pos_kd;
3     float pos_i_limit, pos_output_limit;
4     float vel_kp, vel_ki, vel_kd;
5     float vel_i_limit, vel_output_limit;
6     float max_v, max_a, mass, drag;
7 };
8
9 struct ChassisConfig {
10     bool planner_enabled;
11     AxisConfig x, y, z, roll, pitch, yaw;
12 };

```

运行时通过 /zit6/update_params 服务实时重载。

8.2 全局配置实例

Listing 10: 全局配置访问

```

1 namespace auv::config {
2     extern SystemConfig sys_config; // 只读运行时配置
3     extern const ParamMeta SYSTEM_PARAMS[]; // 参数注册表
4     extern const size_t SYSTEM_PARAMS_COUNT;
5 }

```

9 坐标系定义

采用两套坐标系，轴向定义不同：

坐标系	约定	X/Y/Z	原点
机体系	FRD	前/右/下	AUV 重心
世界系	NED	北/东/下	INS 上电零点

表 5: 坐标系定义

6-DOF 全量运动学变换：

$$\nu_{world} = R(\phi, \theta, \psi) \cdot \nu_{body} \quad (1)$$

其中 $R(\eta)$ 为 6×6 分块矩阵:

$$R(\eta) = \begin{bmatrix} R_{ZYX}(\phi, \theta, \psi) & 0_{3 \times 3} \\ 0_{3 \times 3} & T(\phi, \theta) \end{bmatrix} \quad (2)$$

10 仿真支持

10.1 SITL (Software-in-the-Loop)

- 通过 `/zit6/sim/nav` 话题接收仿真导航数据
- 队列长度 3, 消费端 drain 到最新帧
- 可在无硬件条件下测试控制逻辑

10.2 HITL (Hardware-in-the-Loop)

- `HitlSimulator` 类在固件内部模拟 AUV 动力学
- 每步积分推进器推力, 更新位姿
- 用于推进器分配和 PID 参数调优

仿真模式切换通过 `config.json` 中的 `simulation` 字段控制。